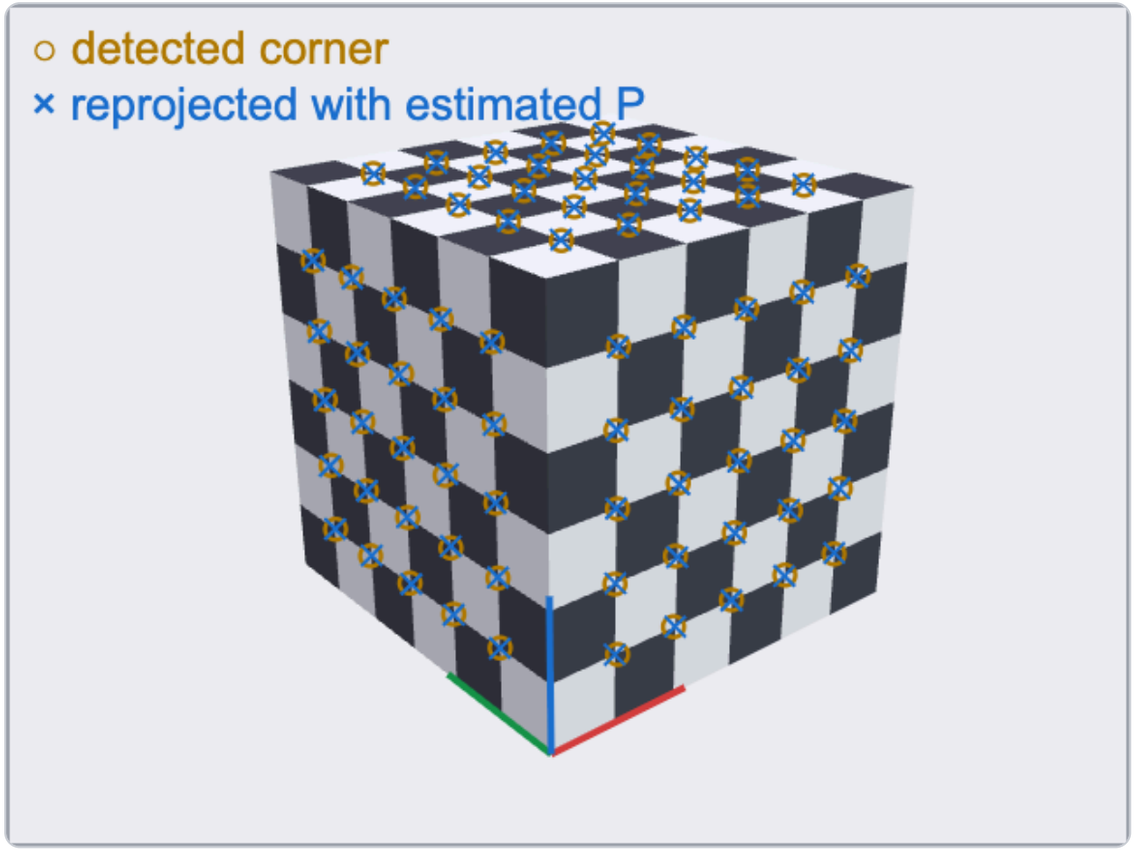
Lecture 4
3D scene reconstruction & SLAM

Robot Control · University of Warsaw

Structure from motion, bundle adjustment, a look at NeRFs and Gaussian splatting, and simultaneous localisation and mapping.

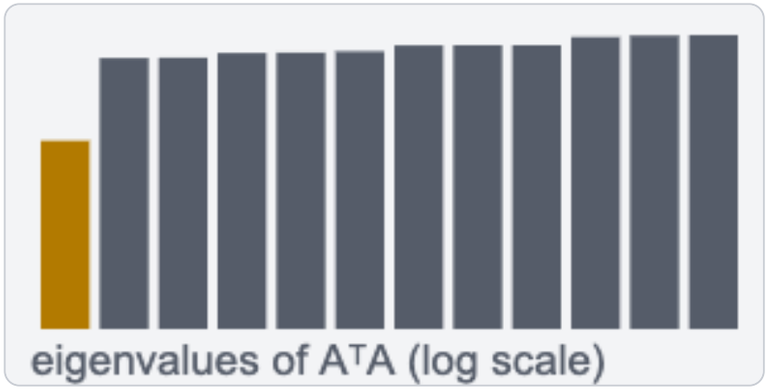
Previously on Robot Control…

Calibration + PnP: known object → determine camera parameters

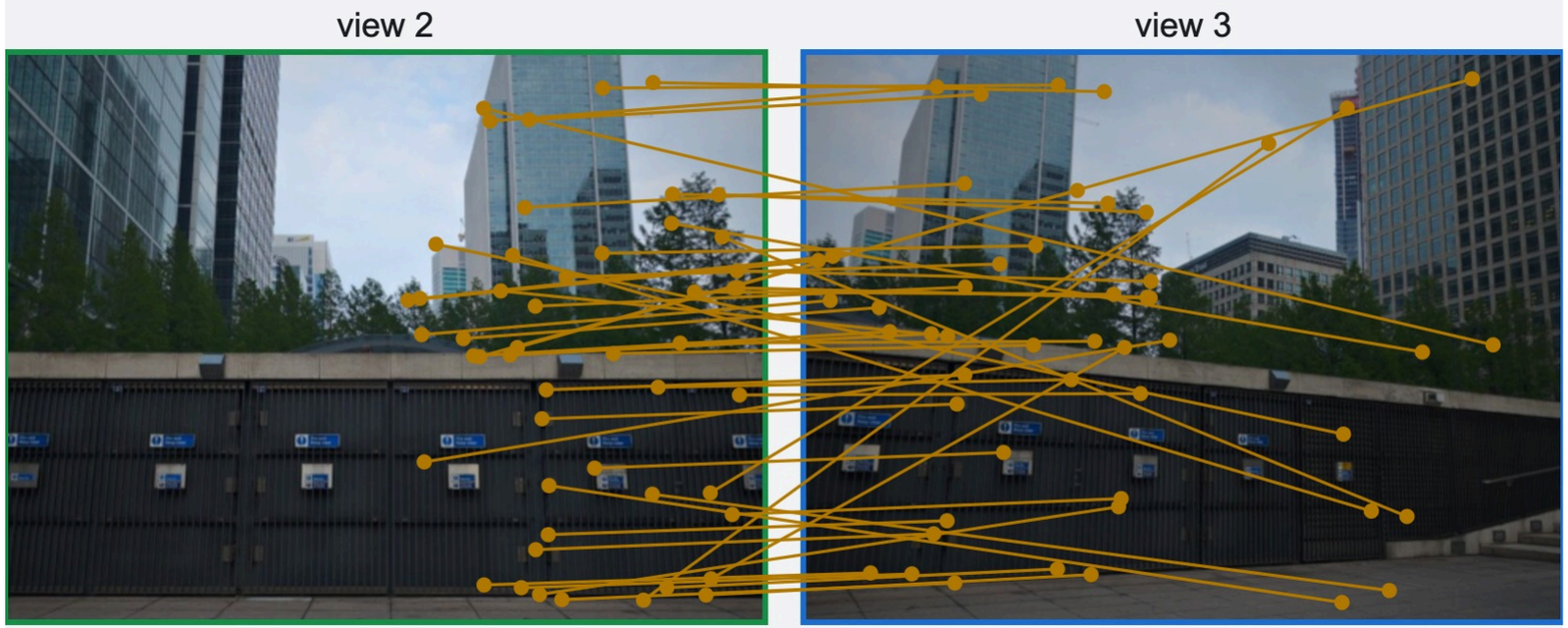


	true	estimate
f_x	800	788
f_y	800	789
o_x	330	330
o_y	235	237
skew s	0	0.2
camera pos. error	0	0.25

75 points · RMS reprojection error
0.73 px



Matches + RANSAC: determine correspondence between two images



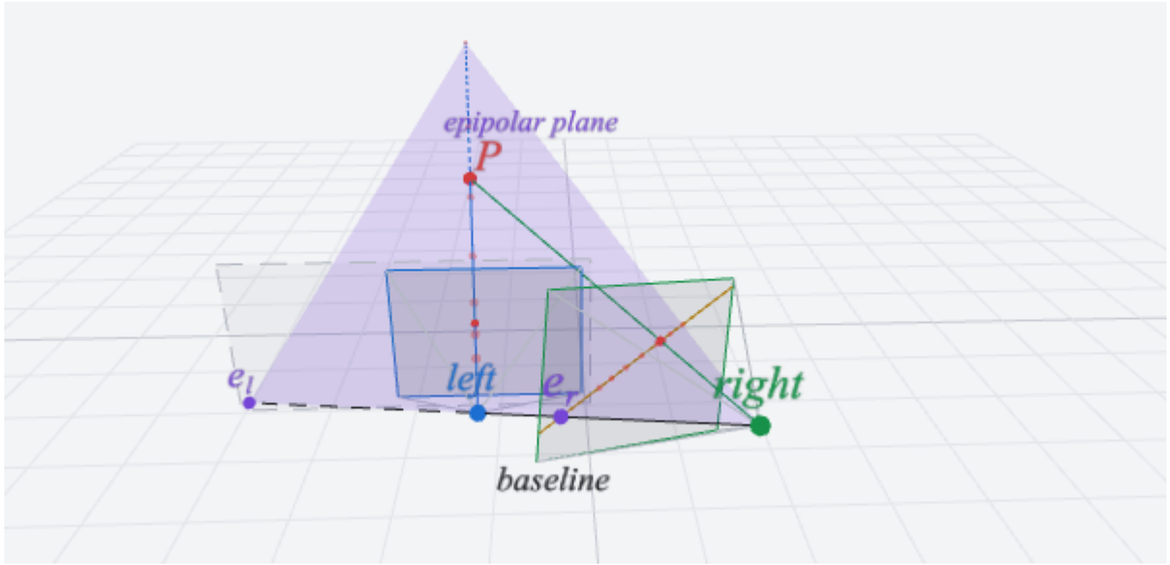
50 matches between view 2 and view 3.

From a feature matcher: most are right, but 15 (30%) are wrong. Which ones?

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Two-view geometry: from matches to relative poses and 3D points

Why it matters: matching becomes a 1D search along the line, and wrong matches can be rejected; the line depends only on the relative pose $R, \mathbf{t}$ (and K), so every correct match is a **measurement of the camera motion**.





$\tilde{\mathbf{u}}_l = (280,\ 200,\ 1)$

epipolar line $\mathbf{l}_r = F\tilde{\mathbf{u}}_l$:

$0.505\,u + 0.863\,v - 344.5 = 0$

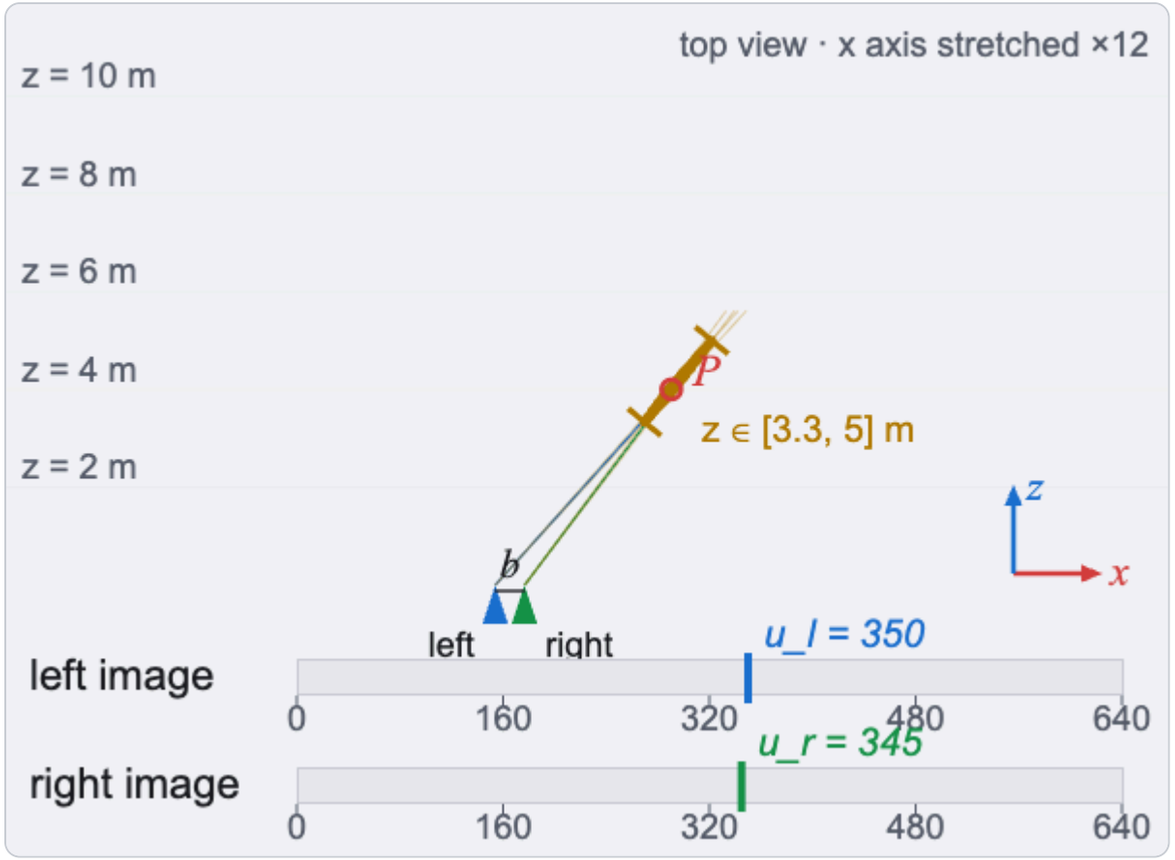
P in the right image:

$\tilde{\mathbf{u}}_r = (387.5,\ 172.4,\ 1)$

$\tilde{\mathbf{u}}_r^\top F\,\tilde{\mathbf{u}}_l = 0$ for every depth

epipole $e_r = (68,\ 359)$

Stereo vision: depth from two cameras, and its uncertainty



$$u_l = f\,x/z + o_x = 350$$

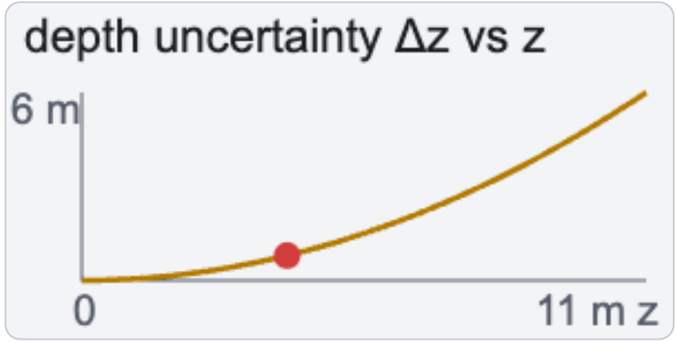
$$u_r = f\,(x - b)/z + o_x = 345$$

$$d = u_l - u_r = 5 \text{ px}$$

$$z = \frac{b\,f}{d} = \frac{0.05 \cdot 400}{5} = 4 \text{ m}$$

$$\Delta z \approx \frac{z^2}{bf}\,\Delta d = 0.8 \text{ m}$$

$\Delta d = 2e = 1$ px (each pixel ± 0.5)

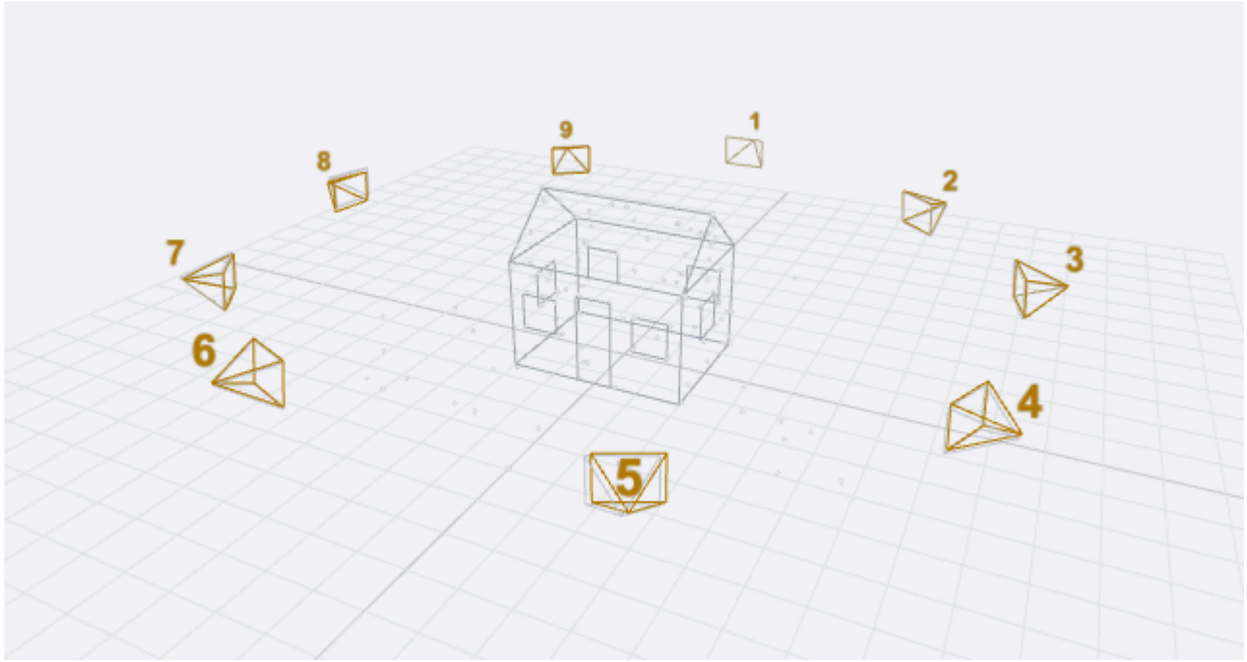


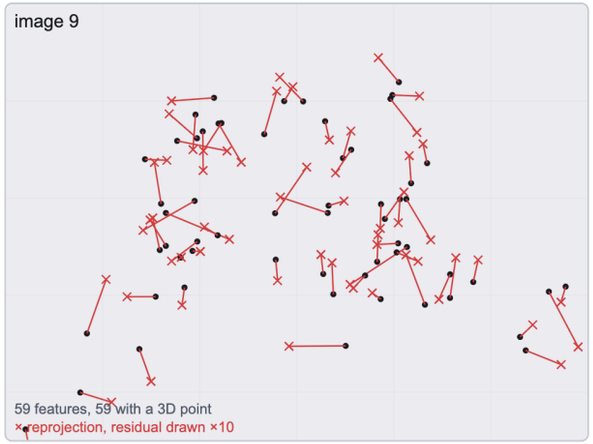
Drag P.

Today

Combine everything

Part 1: 3D Scene Reconstruction





19/19 **global bundle adjustment: all 9 cameras, 124 points**
cameras **9**/9 · points **124** · observations **548**
unknowns 6·9 + 3·124 = **426** (7 fixed: gauge), equations 2·548 = **1096**
RMS reprojection error 5.47 px → **2.73 px**

Toy reconstruction of a house from photos.

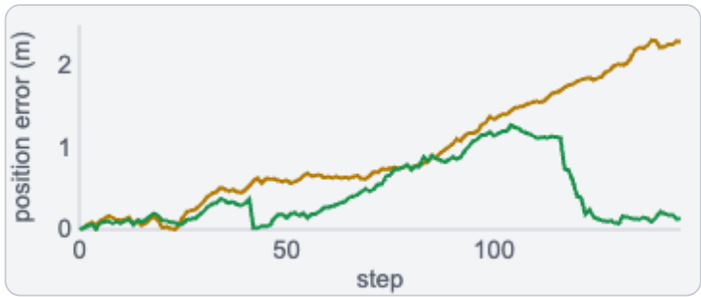
- **(lecture 3)** We start from one pair,
- **(lecture 2)** Then register one camera at a time with PnP and triangulate the points the new camera sees,
- **(today)** after every few images we adjust **everything** so that all reprojections agree: **bundle adjustment**.

The residuals (red) in the current image show the error that is minimised.

Part 2: Simultaneous Localisation and Mapping (SLAM)



step **145** / 145 · **55 m** driven · sees 0
dead reckoning **2.3 m** off, 7.5°
SLAM **0.14 m** off, 0.3°
map: **14** / 14 landmarks, 0.13 m off
463 unknowns, 331 factors
past the start: the loop is closed



-- true path, △ robot, × true landmarks
— ▲ dead reckoning (odometry only)
— ▲ ● SLAM: poses, map, ○ 95% regions
◢ sensor: range and field of view
graph: ● poses ■ landmarks — odometry ／
observation ▲ prior

A robot drives around a block, knowing roughly how far its wheels turned (orange: the errors add up, as with the chained camera) and seeing a few landmarks near it. Without a map it cannot tell where it is; without knowing where it is, it cannot place the landmarks. Doing **both at once**, with the same kind of adjustment as on the previous slide, gives the green path and map: when the first landmarks reappear, everything snaps into place.

Plan: 1. from photos to a 3D model: structure from motion, bundle adjustment, a glance at NeRFs and Gaussian splats · 2. the same idea on a moving robot: SLAM.

Part 1

3D Scene Reconstruction

Structure from Motion (SfM)

General Idea

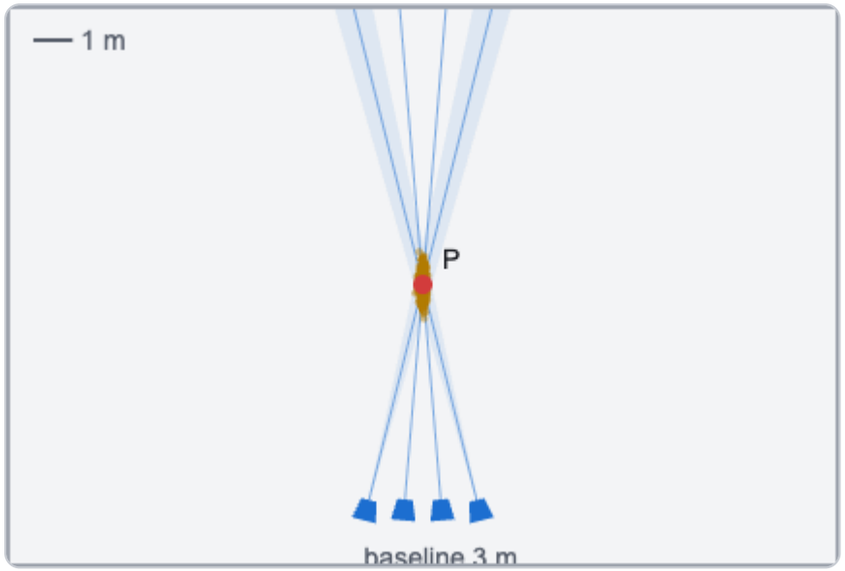
video: youtube.com/watch?v=i7ierVkXYa8

An application: NeRFs and Gaussian splatting

Photo-real 3D scenes from ordinary photos: what we cover today is the basis of both.

video: youtube.com/watch?v=9jcl6R7leFE

Why many views?



4 cameras, σ = 1°, Z = 6 m
uncertainty of P (1σ): depth **29.7 cm**, sideways **5.4 cm**
250 estimates from noisy bearings (dots), 95% ellipse (dashed)

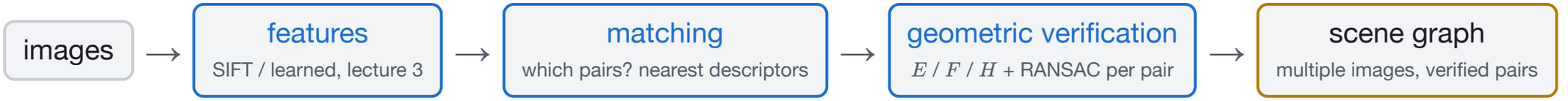
One point, seen from N cameras. Every observation is noisy*, so the estimated rays do not meet perfectly. We use least-squares to find the (approximate) solution.

- more views: the uncertainty shrinks like $1/\sqrt{N}$
- a wider baseline helps much more: the depth uncertainty is $\propto Z^2/B$ (lecture 2) - intuition: rays that are parallel don't help
- every view also constrains the cameras: with enough points in common, the poses and even the intrinsics follow from the photos alone (self-calibration)
- coverage: parts hidden in one photo are seen in another; wrong matches are outvoted

* Some uncertainty is inevitable: a feature is located to about half a pixel at best, and which pixel a photon lands in is random.

The Structure from Motion (SfM) pipeline

1. Correspondence search: which images see the same things, and where



2. Reconstruction: grow the model one image at a time, keep it consistent



- green and blue boxes: tools we already know
- yellow boxes: what we cover today

Output:

- the cameras: intrinsics and pose of every image
- the 3D points: position, colour, and the list of images each one is seen in
- a reprojection error for every observation: how well the model explains it

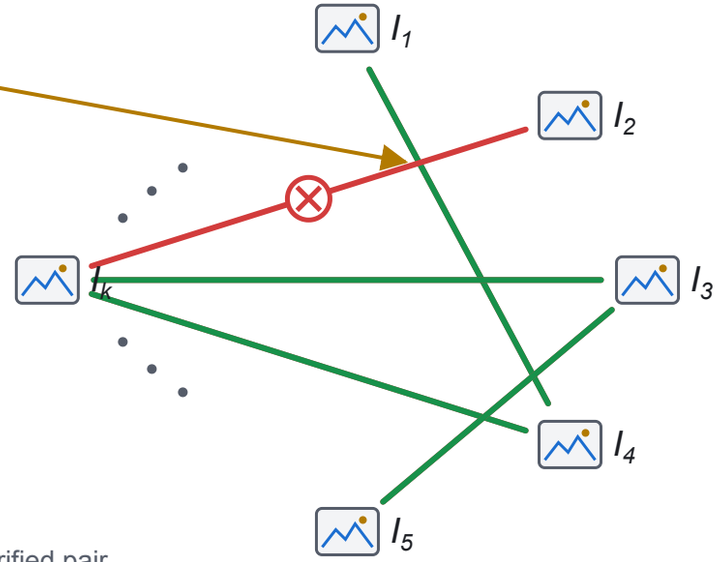
The pipeline of COLMAP (Schönberger & Frahm, Structure-from-Motion Revisited, 2016; colmap.github.io).

Correspondence search

1. **Feature extraction** → in COLMAP it is SIFT (lecture 3)
2. **Matching** → images with many matched features form a **potential** pair
3. **Geometric verification** → for each potential pair, estimate the transformation between the two images; if enough matches agree with it, the pair is **verified**

Many matches → potentially a pair…

… estimate the camera transformation (epipolar geometry + RANSAC, lecture 3) …



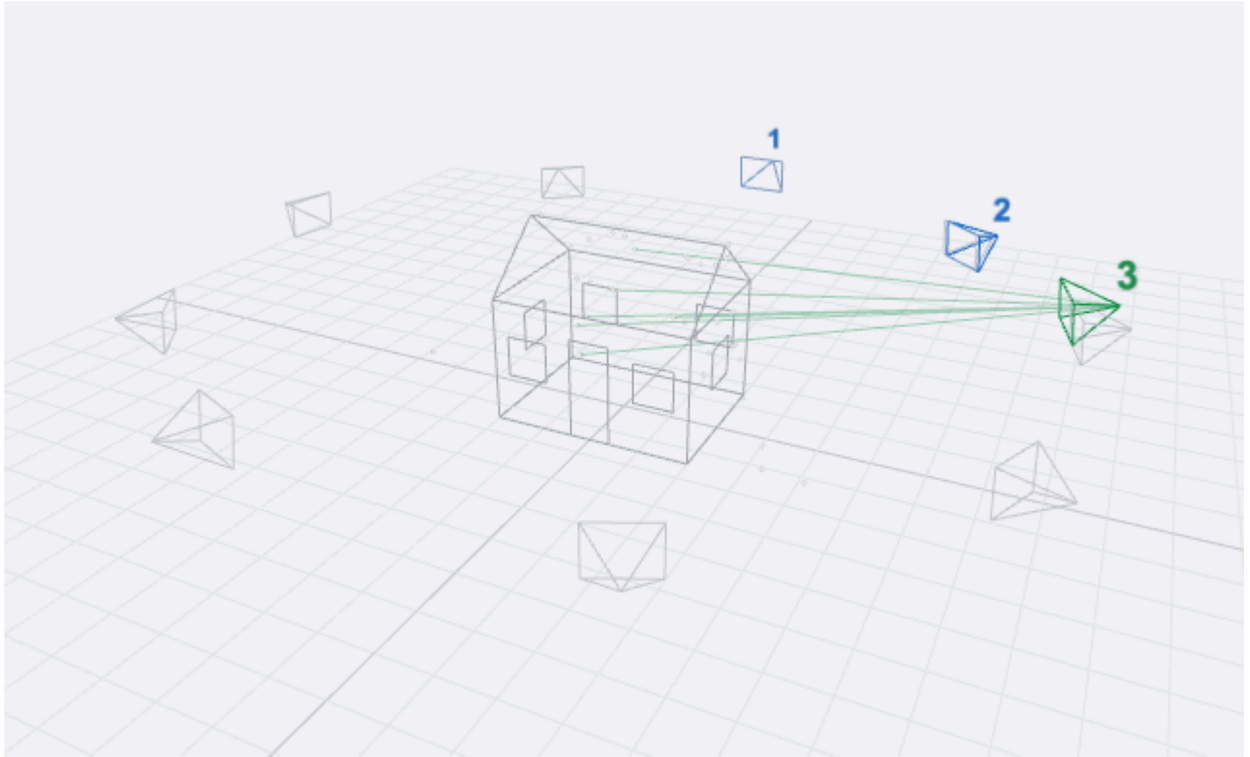
… use it to reproject the features of one image into the other …

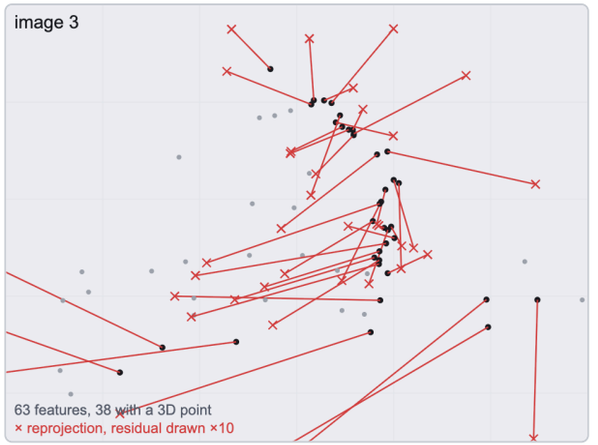
… good fit → pair verified, otherwise throw the pair out

edge = the two images overlap: red = potential pair, green = verified pair

> At the end we have **pairs of images**, and for each pair the **geometric relation** between its two images.

Reconstruction, step by step






```
3/19 register camera 3: PnP
on 6 known points
cameras 3/9 · points 40 ·
observations 118
unknowns 6·3 + 3·40 = 138 (7
fixed: gauge), equations
2·118 = 236
RMS reprojection error 7.19
px
```

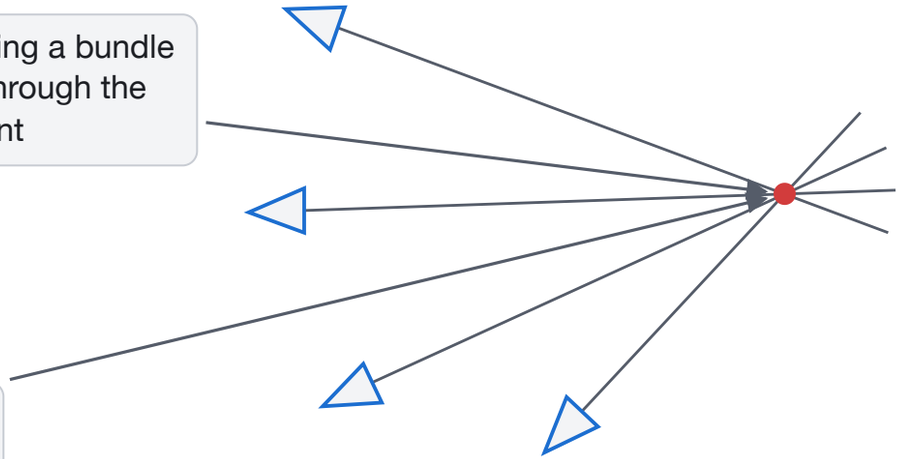
Use "next step": each new camera is registered with PnP against the points we already have (green rays), the points it sees are triangulated, and after every few images a bundle adjustment pulls all cameras and points together (orange: what moved). Bundle adjustment: next.

Reconstruction: bundle adjustment

- previous lectures: two images and one point
- most often we have more…
- **bundle adjustment**: use multiple views and multiple points



Name origin: adjusting a bundle of rays crossing through the same point



More precise info (e.g. different resolution)

Allows to deal with noise in the data

> The rays through one point form a **bundle**. Adjusting all cameras and all points until every bundle agrees: **bundle adjustment**.

Bundle adjustment, mathematically

$$\pi_j\left(\mathbf{P}_j,\ \mathbf{X}_i\right) = \mathbf{u}_{i,j}$$

Known (observed, so noisy) 2D projection of the point

Estimated reprojection function, e.g. a projection matrix, but can also be more general, e.g. include undistortion

Estimated camera pose

Estimated 3D point coordinates

$$j = 1, \ldots, J$$

cameras / images

$$i = 1, \ldots, I_j$$

features seen in a given image

Computational complexity of bundle adjustment

Example with known camera calibration parameters

10 k images

~1 k features per image

each 3D point seen in ~10 images

$$\pi_j(\mathbf{P}_j, \mathbf{X}_i) = \mathbf{u}_{i,j}$$

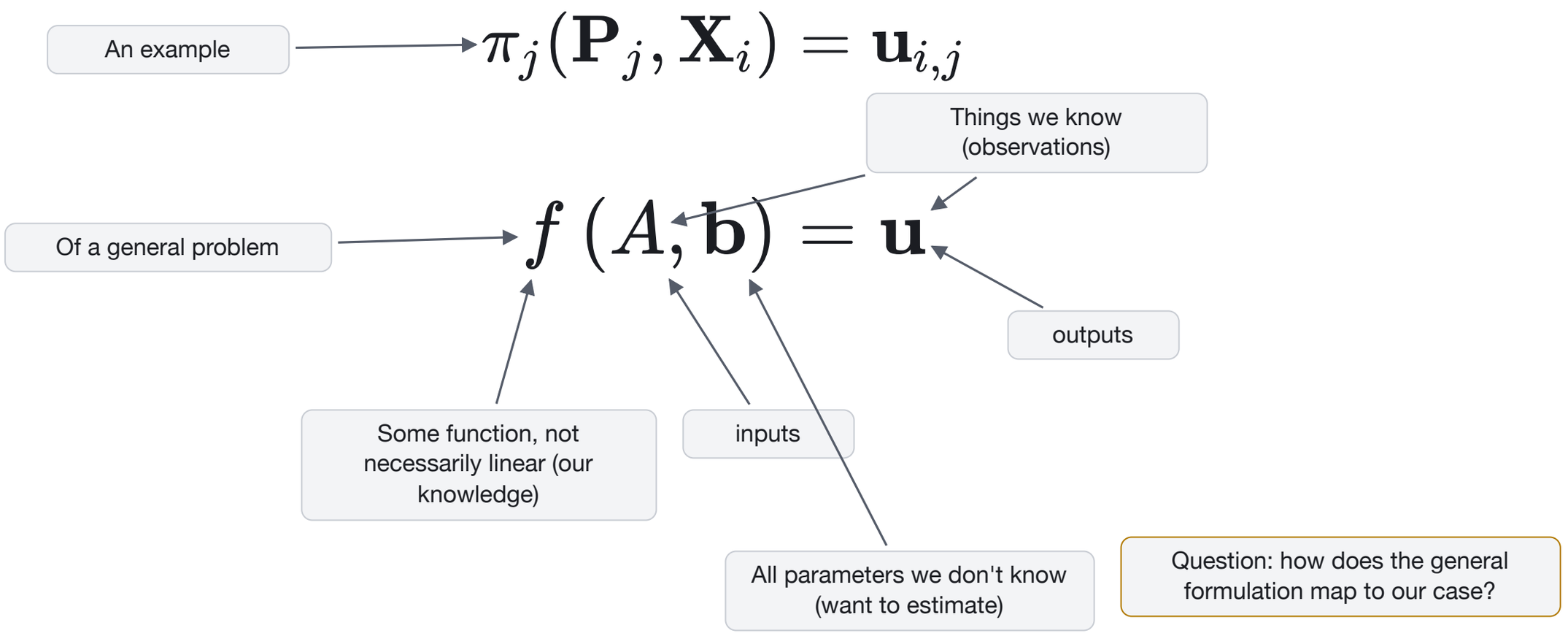
Constraints: 2 × 1 k × 10 k = **20 M** (pixel coordinates × #features per image × #images)

Unknowns: 3 DoF × #3D points + 6 DoF × #cameras = **3 M** + **60 k**

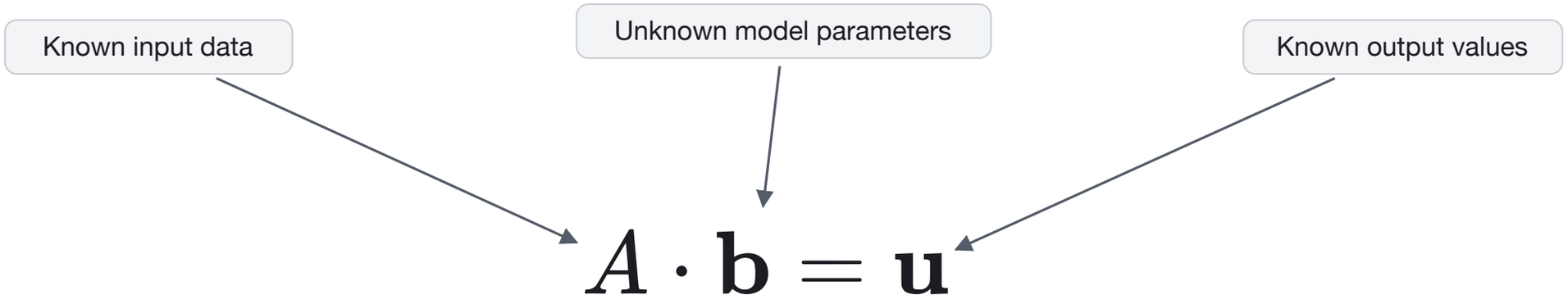
10 k images × 1 k features = 10 M observations, but on average 10 observations correspond to the same 3D point → 1 M points

So more constraints than unknowns!

Sketch of finding the solution



Example: linear regression

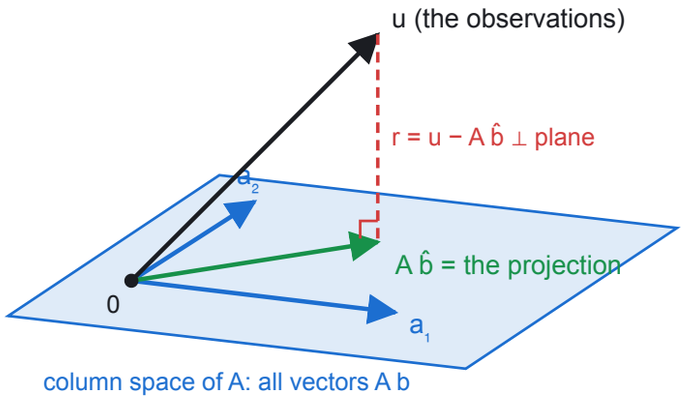


$$x_1\beta_1 + \ldots + x_k\beta_k + 1 \cdot \beta_{k+1} = y$$

How do we solve it if there are more constraints than unknowns (usually true)?

Think about the geometrical interpretation…

Linear regression: least squares



$$A \cdot \mathbf{b} = [\mathbf{a}_1, \ldots, \mathbf{a}_k] \cdot \mathbf{b} = \mathbf{u}$$

the columns span a subspace; $\mathbf{b}$ picks a combination of them; with more constraints than columns, $\mathbf{u}$ is usually not in the subspace

$$d(\mathbf{b}) = \|A\mathbf{b} - \mathbf{u}\|^2$$

we can't get $\mathbf{u}$, so the next best thing is the closest point in the subspace, i.e. minimise the (squared) distance

$$d(\mathbf{b} + \mathbf{h}) - d(\mathbf{b}) = 2\,(A\mathbf{b} - \mathbf{u})^\top A\,\mathbf{h} + \mathbf{h}^\top A^\top A\,\mathbf{h}$$

linear term → the first derivative

$$(A\mathbf{b} - \mathbf{u})^\top A = 0 \iff A^\top A\,\mathbf{b} = A^\top \mathbf{u}$$

$A^\top A$: squared volume of the parallelepiped spanned by the columns of A → invertible if the columns are linearly independent

$$\mathbf{b} = (A^\top A)^{-1} A^\top \mathbf{u}$$

with these coefficients we get the desired projection

> This is **least squares**, i.e. the closest distance!

What if the problem is nonlinear?

Initialise with some parameters (the current model)

$$\| f(A, \mathbf{b}) - \mathbf{u} \|^2$$

Taylor expand to convert it into a linear problem:

$$\mathbf{R} = \mathbf{u} - f(A, \mathbf{b}) = f(A, \mathbf{b} + \Delta\mathbf{b}) - f(A, \mathbf{b}) \approx D_{\mathbf{b}} f(A, \mathbf{b})\, \Delta\mathbf{b}$$

Current error

The value we would like to have

The value we get from the current model

green: we know it · red: unknown

$$\mathbf{R} \approx D_{\mathbf{b}} f(A, \mathbf{b})\, \Delta\mathbf{b}$$

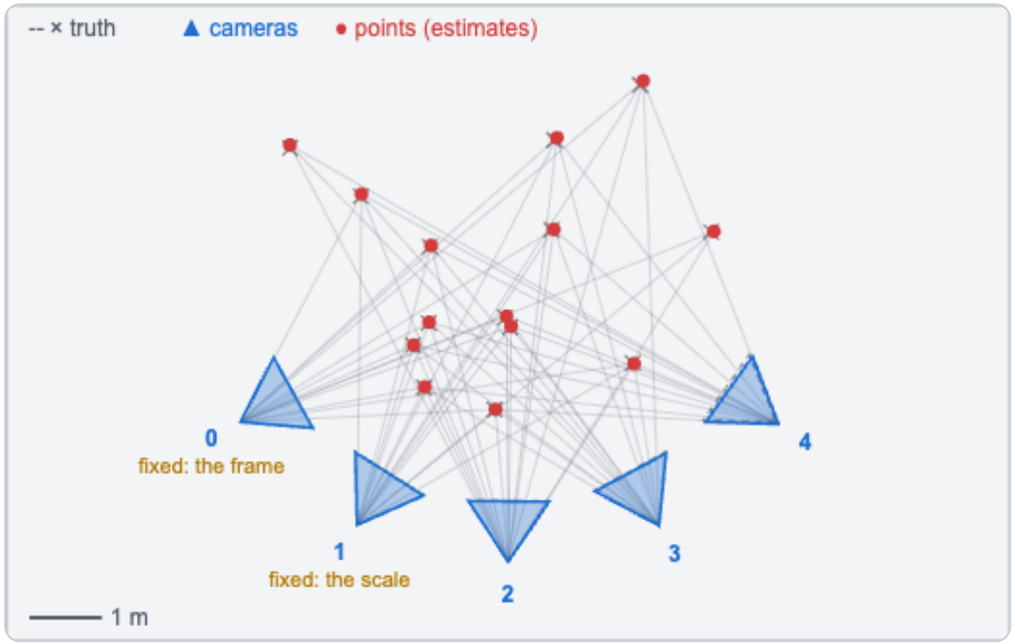
Linear!

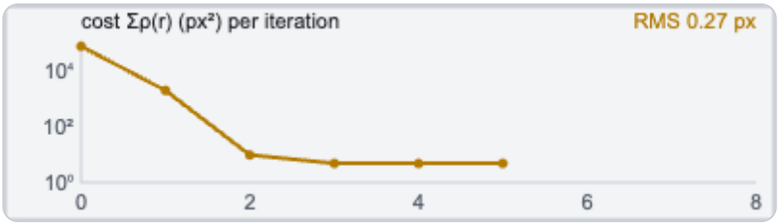
Do we know everything we need? Yes

Solve for $\Delta\mathbf{b}$, move, and then…

ITERATE and hope for the best…

Bundle adjustment to play with (in 2D)



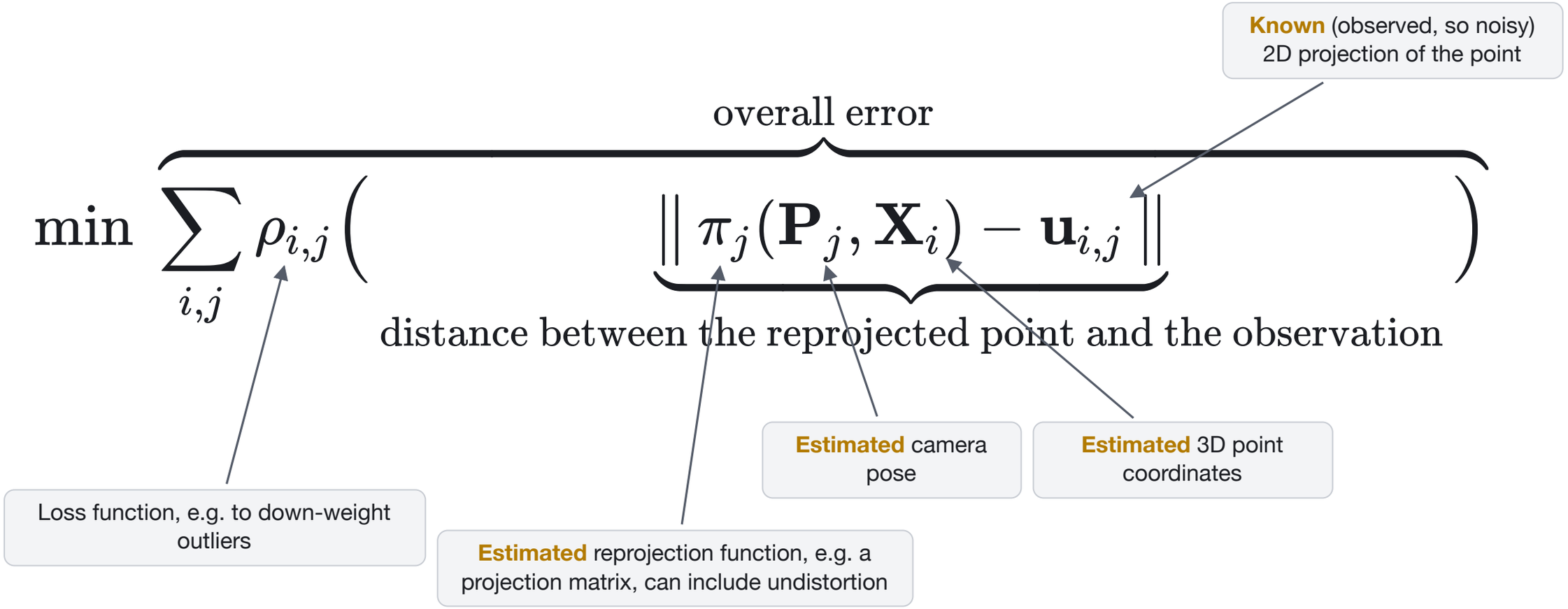



```
J = 5 cameras, N = 14 points, M = 66
observations
RMS reprojection error 0.27 px
converged, 5 steps
```



A flat world: cameras with a 1D image (one pixel coordinate per observation), points in the plane. The estimate starts perturbed; every step linearises, solves and moves all cameras and points at once, and the residuals (the ticks on the 1D images) shrink together. Camera 0 and the distance to camera 1 are held fixed: the images do not say where the whole model is or how big it is.

The optimisation formula for bundle adjustment



How do we deal with the computational complexity?

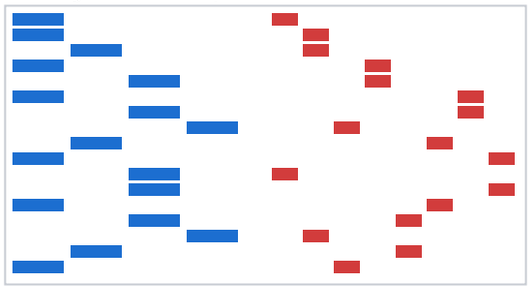
$$\min \sum_{i,j} \rho_{i,j} \left(\left\| \pi_j(\mathbf{P}_j, \mathbf{X}_i) - \mathbf{u}_{i,j} \right\| \right)$$

$$\mathbf{R} \approx D_{\mathbf{b}} f(A, \mathbf{b}) \, \Delta \mathbf{b}$$

What can we say about this matrix in our problem?

It has lots of zeros, AKA it is **sparse**!

J = D_b f : one row per observation

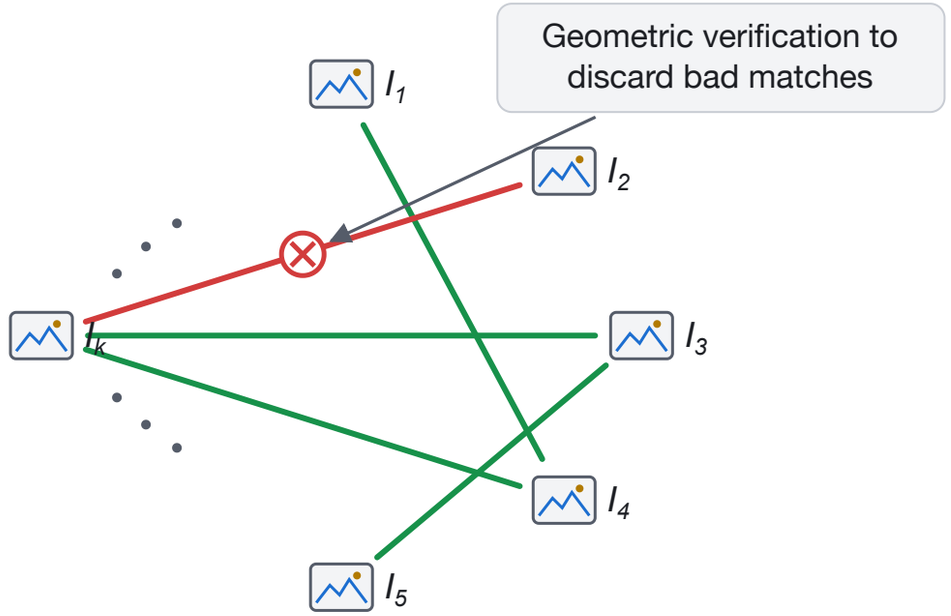


camera columns point columns

a row has entries only for its own camera and its own point: everything else is zero

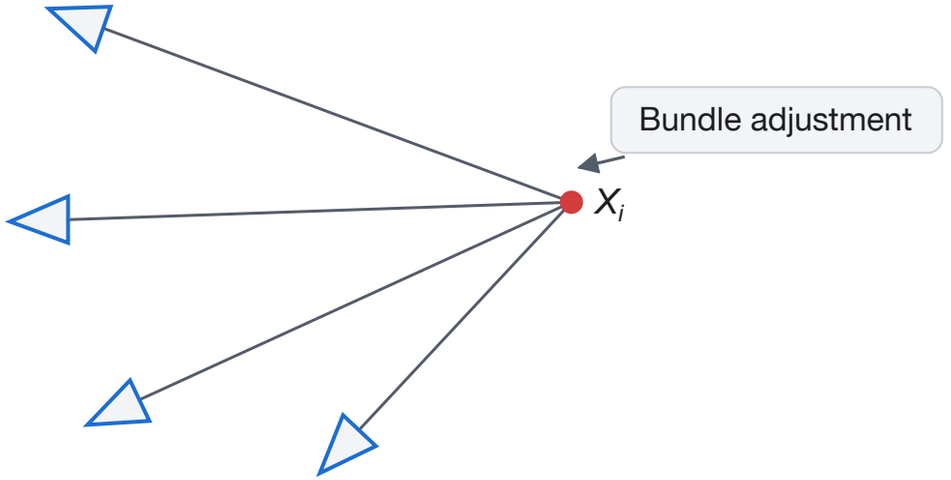
Structure from motion: summary

I. Correspondence search: which images overlap



edge = the two images overlap

II. Reconstruction: estimate poses for cameras and 3D points using info from all images



cameras and points: adjusted together

- find the projections using the current estimates (the current beliefs = the scene model)
- compare with the observations (the ground truth) to compute the error
- adjust iteratively to minimise it

COLMAP: state-of-the-art SfM

- works on datasets built from images collected from the internet
  - so different cameras, not necessarily calibrated
  - potentially very large datasets
- COLMAP: Structure-from-Motion and Multi-View Stereo, colmap.github.io

Some recent results: VGGSfM, Visual Geometry Grounded Deep Structure From Motion (2024), vggsfm.github.io: the whole pipeline, from matching to bundle adjustment, learned end to end.

Part 2

SLAM

simultaneous localisation and mapping: the same idea, on a moving robot

SLAM from a camera

One camera. Top: the features tracked in the current image (lecture 3). Bottom: the map of points and the trajectory, grown as the camera moves.

video: youtube.com/watch?v=ufvPS5wJAx0

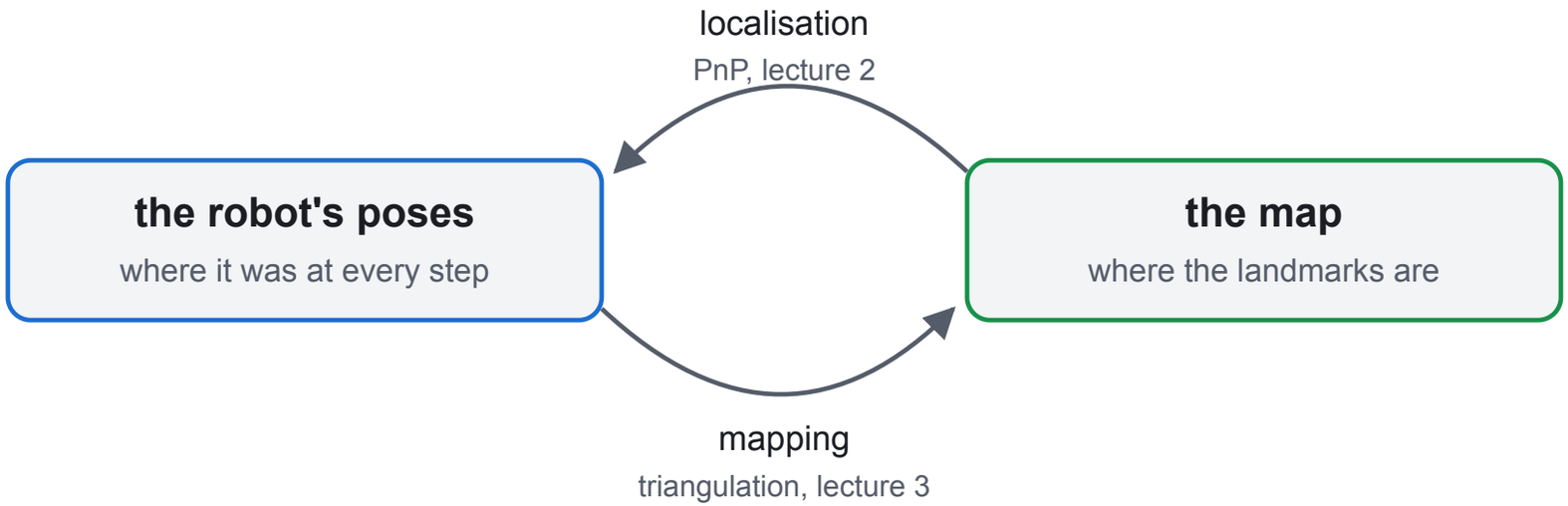
SLAM with a neural map

One camera again, but the map is a NeRF, built in real time while the camera moves.

video: youtube.com/watch?v=-6ufRJugcEU

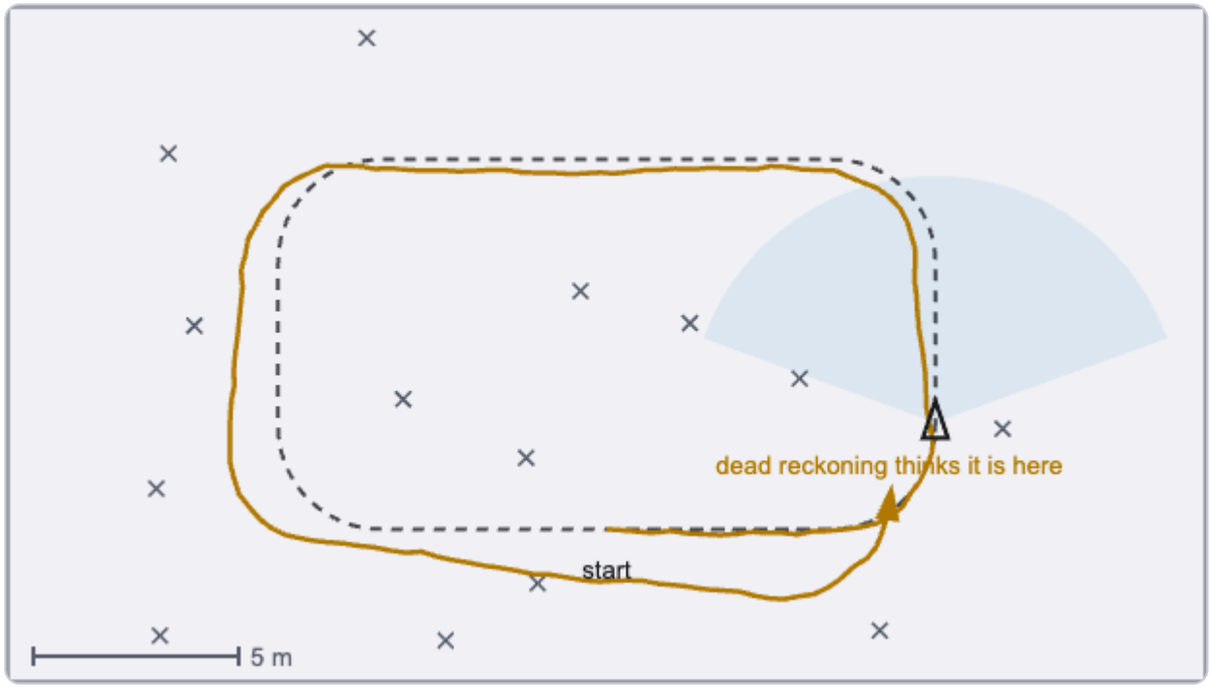
SLAM: the problem

A robot in an unknown environment needs its **pose** (to control itself) and a **map** (to plan). Each helps with the other is known:

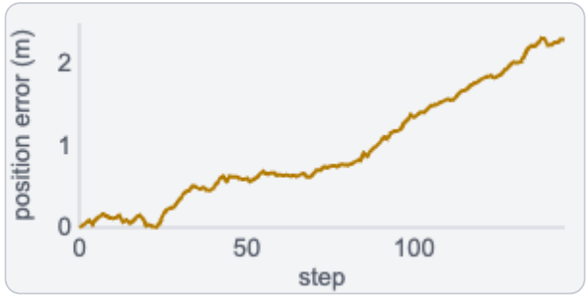


> **SLAM**: neither is known. Estimate the poses and the map **together**, from noisy motion and noisy observations.

Dead reckoning: the robot without a map



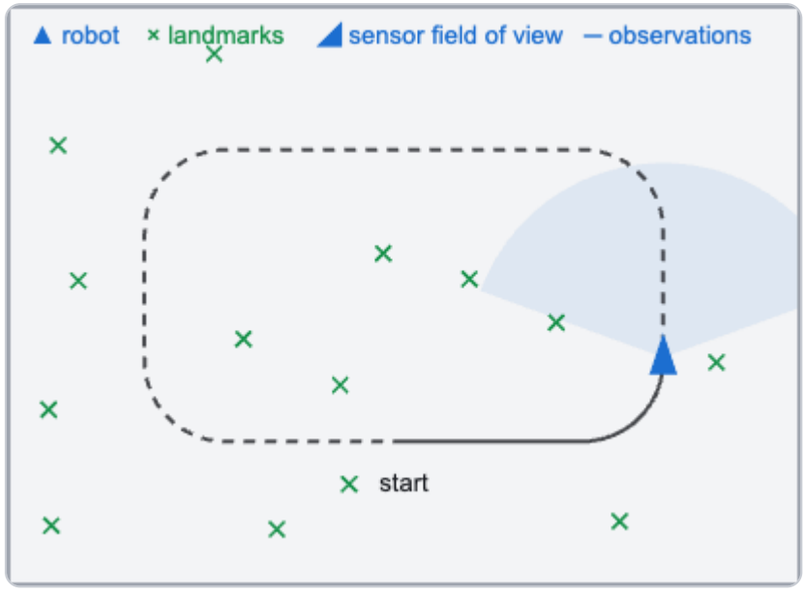
step **145** / 145 · **55 m** driven · sees 0
dead reckoning **2.3 m** off, 7.5°
odometry only: errors add up, nothing removes them



-- true path, △ robot, × true landmarks
— ▲ dead reckoning (odometry only)
— ▲ ● SLAM: poses, map, ○ 95% regions
◢ sensor: range and field of view
graph: ● poses ■ landmarks — odometry ／ observation ▲ prior

Dead reckoning adds up the odometry. Each step adds a small, independent random error, so the estimate performs a **random walk**: its spread grows like $\sqrt{\text{number of steps}}$. An error in the estimated direction of travel (the heading θ) impacts also position estimates: every later step is added in a wrong direction, so the error grows with the distance driven. And if the chain breaks (the robot is carried, the sensor is covered), it is lost for good: with no map, there is nothing to find itself with.

What we would like to have



Unknown too, but we already have an estimate of it from the previous step. Both poses keep being adjusted.

$$\mathbf{z}_t = f\left(\mathbf{x}_{t-1}, \mathbf{x}_t\right)$$

Measured: the robot's motion since the last step (noisy)
e.g. from the wheels, or from the images (visual odometry*)

the motion that the two poses imply

Unknown: the robot's pose at step t: where it is and which way it faces

$$\mathbf{z}_{t,k} = h\left(\mathbf{x}_t, \mathbf{l}_k\right)$$

Measured: the distance (range) and the direction (bearing) to landmark k (noisy)
e.g. from a laser scanner; a camera gives the pixel instead

what the sensor should see from $\mathbf{x}_t$ if the landmark is at $\mathbf{l}_k$

Unknown: where landmark k is

> **With a camera as the sensor, this is Part 1 again**: the landmarks are 3D points, $\mathbf{z}_{t,k}$ is the pixel where point k appears in image t, and h is the projection π. SLAM is bundle adjustment running on the robot.

* odometry: from Greek hodós, road, and métron, measure

Can't solve directly so we define the cost function

$$\min_{\mathbf{x}_{0:T},\, \mathbf{l}_{1:K}} \frac{\left\|\mathbf{x}_0 - \bar{\mathbf{x}}_0\right\|^2}{\sigma_0^2} + \sum_{t=1}^{T} \frac{\left\|f(\mathbf{x}_{t-1}, \mathbf{x}_t) - \mathbf{z}_t\right\|^2}{\sigma_{\text{odo}}^2} + \sum_{(t,k)} \frac{\left\|h(\mathbf{x}_t, \mathbf{l}_k) - \mathbf{z}_{t,k}\right\|^2}{\sigma_{\text{obs}}^2}$$

prior: estimated starting pose is what we measured at the beginning

odometry: consecutive poses should agree with the measured motion between them

observations: every landmark should be where the robot saw it from

weights $1/\sigma^2$: a measurement counts according to its noise; when measurements disagree, the precise ones win over the noisy ones.

> The same recipe as bundle adjustment: for every measurement, (what it should be, given the unknowns) – (what it was), squared, weighted, summed. Then linearise, solve, move, iterate.

Where the cost comes from

$$e_i = z_i - h_i(\mathbf{x}) \sim \mathcal{N}(0, \sigma_i^2)$$

the error of measurement i: what it was, minus what it should be given the unknowns $\mathbf{x}$

Assumption: the noise is Gaussian: usually small, rarely large; σ_i says how noisy

$$p(e_1, \ldots, e_n \mid \mathbf{x}) = \prod_i \frac{1}{\sqrt{2\pi}\,\sigma_i} \exp\left(-\frac{e_i^2}{2\sigma_i^2}\right)$$

how likely are the errors we see, given $\mathbf{x}$? The best $\mathbf{x}$ makes them most likely

independent measurements: a product of bell curves

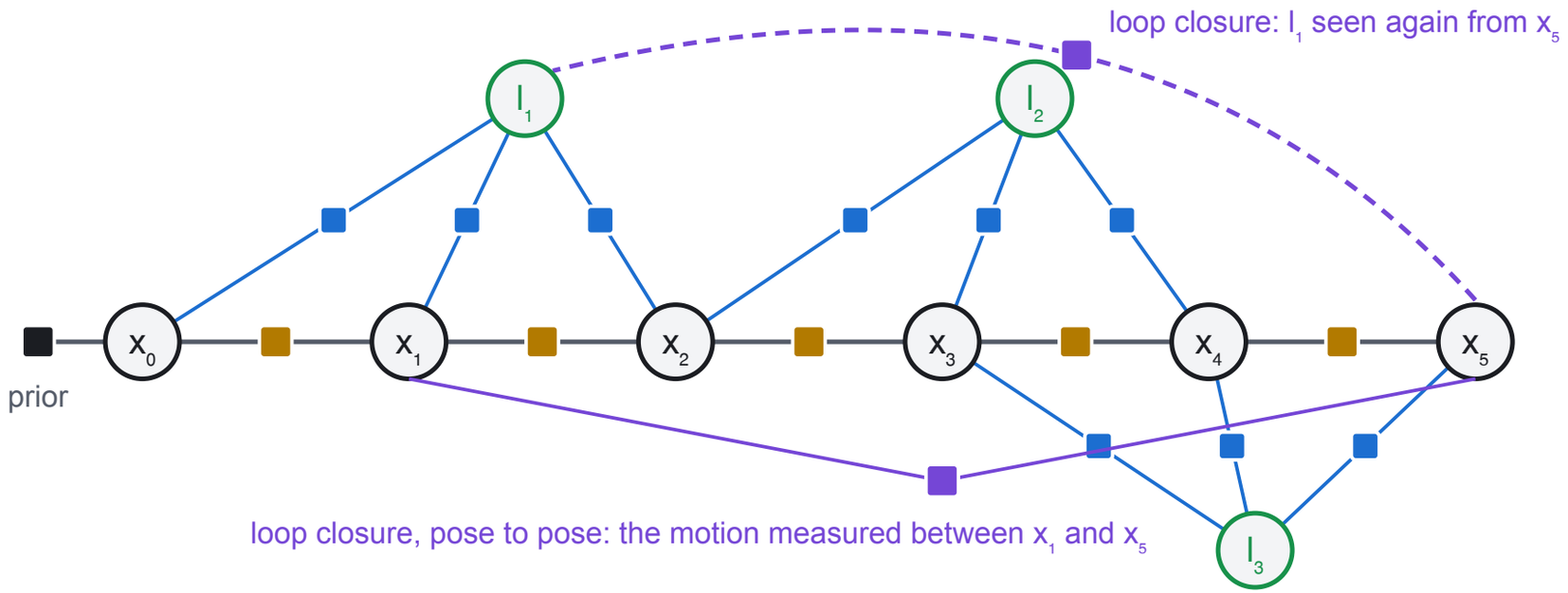
$$\arg\max_{\mathbf{x}} p = \arg\min_{\mathbf{x}} \sum_i \frac{e_i^2}{\sigma_i^2}$$

the log turns the product into a sum and the max into a min; the constants drop out

the weights $1/\sigma^2$ fall out by themselves: a precise measurement counts more

> Least squares with $1/\sigma^2$ weights is the **most likely** explanation of the measurements, if their noise is Gaussian. That is where the cost comes from.

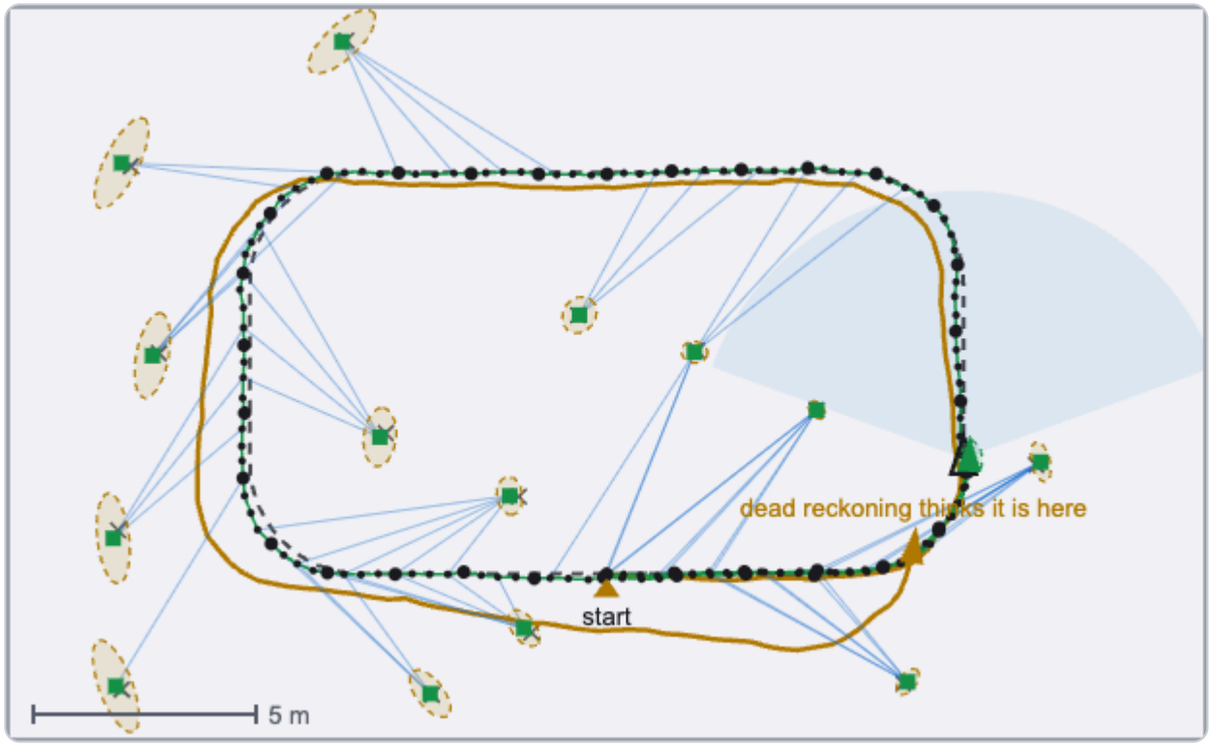
Factor graphs: a picture of the cost



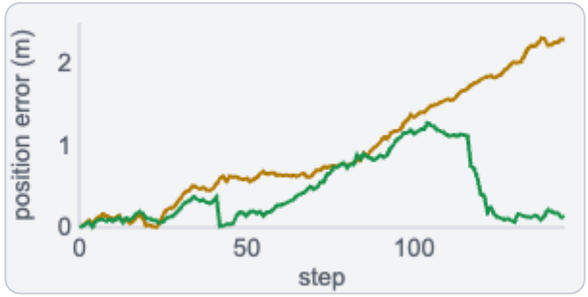
variables: poses x (circles), landmarks l (green circles)
factors (squares): prior, odometry (orange), observation (blue), loop closure between two poses (purple)
cost = Σ over the squares of $\|\text{residual}\|^2/\sigma^2$; the matrix has a non-zero block exactly where a square joins two circles

- circles: the unknowns (poses, landmarks); squares: the terms of the cost (prior, odometry, observations)
- a square touches only the variables in its term: the graph **is** the sparsity pattern, the same zeros as in bundle adjustment
- adding a measurement = adding a square; a **loop closure** = a square between variables far apart in time: a landmark seen again (top), or the motion measured directly between two poses (bottom, as in the animation on the loop-closure slide)

SLAM to play with



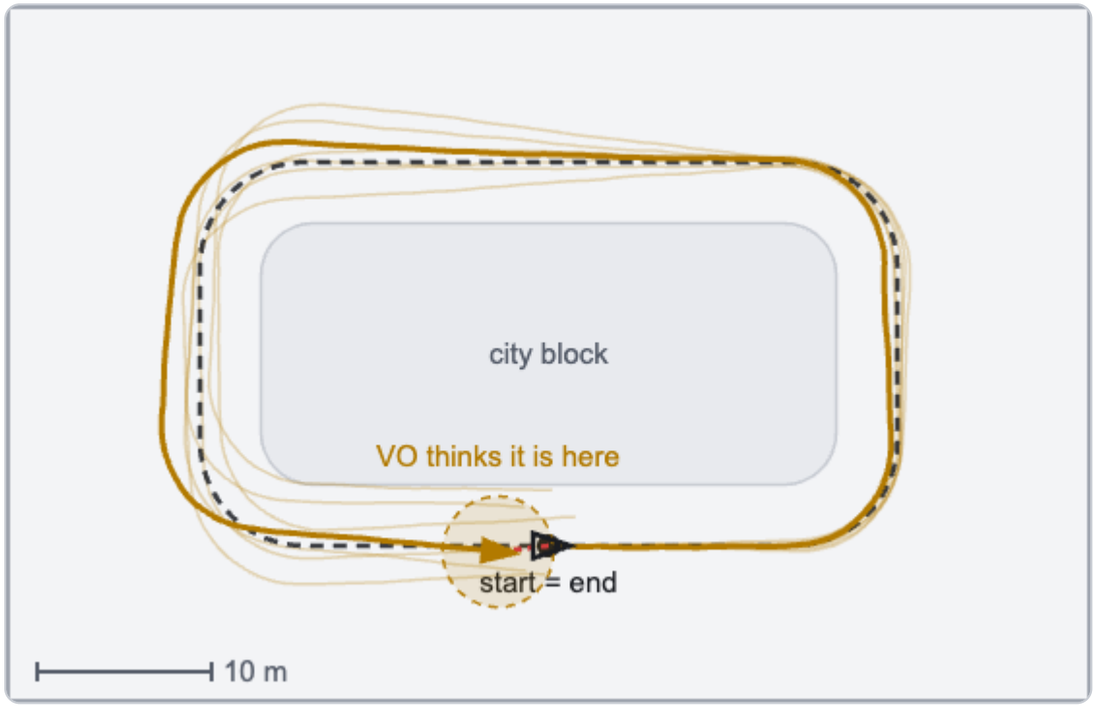
step **145** / 145 · **55 m** driven · sees 0
dead reckoning **2.3 m** off, 7.5°
SLAM **0.14 m** off, 0.3°
map: **14** / 14 landmarks, 0.13 m off
463 unknowns, 331 factors
past the start: the loop is closed



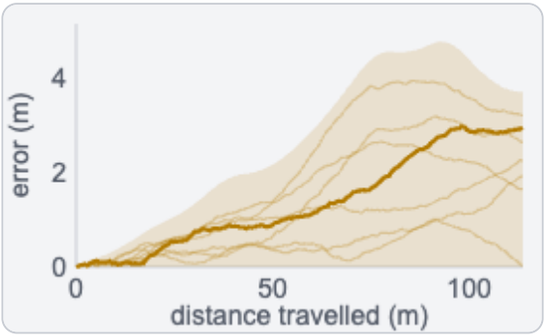
-- true path, △ robot, × true landmarks
— ▲ dead reckoning (odometry only)
— ▲ ● SLAM: poses, map, ○ 95% regions
◢ sensor: range and field of view
graph: ● poses ■ landmarks — odometry ／ observation ▲ prior

The whole graph is re-optimised at every step. Watch the ellipses: the pose uncertainty grows along the lap, the landmarks inherit it, and when the first landmarks are seen again the loop closes: everything moves and the ellipses collapse.

Loop closure



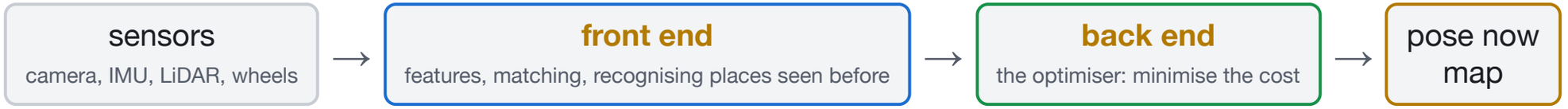
step **160** / 160, driven **114 m**
VO error: **2.9 m**, heading 5.1°
back at the start, but VO does not know it



-- true path, △ true pose
— visual odometry, ▲
faint: other noise draws
◌ 95% uncertainty region
— after loop closure

- odometry and tracking are local: their errors add up along the way (the **drift**)
- recognising a place seen before gives one more measurement, between **now** and **then**
- the optimiser spreads the error around the loop: the whole lap is corrected at once
- a **wrong** loop closure distorts the entire map, so it has to be well verified, like any match (e.g. RANSAC, lecture 3)

Making it run in real time



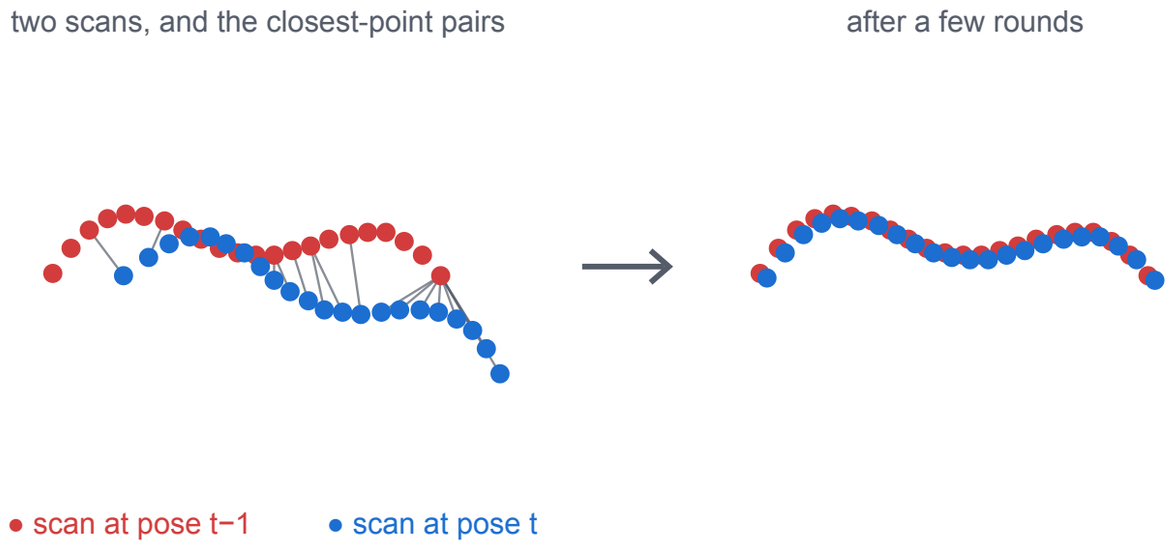
- **front end**: turns sensor data into measurements: features, matching, recognising places seen before (lecture 3)
- **back end**: minimises the cost; the same optimiser as bundle adjustment
- **keep it small**: a **keyframe** only when the view has changed enough, optimise a **window** of recent keyframes, summarise the older ones

Sensors and flavours of SLAM

	what the front end measures	strengths · weaknesses
camera	features → reprojection terms, as in bundle adjustment	cheap, rich · no scale, needs texture and light
stereo / depth camera	metric depth for every frame	scale and dense geometry · short range
camera + IMU	the IMU adds scale, the gravity direction and robustness to fast motion	drones, headsets, phones · the IMU biases must be estimated too
LiDAR	scans aligned to each other and to the map	accurate range to 100 m, works in the dark · cost; tunnels and corridors all look alike

> A front end per sensor, one optimiser: every measurement adds its own terms, with its own σ, to the same cost.

Non-RGB based SLAM example

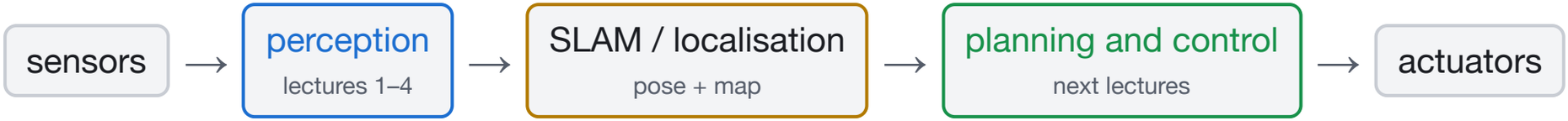


- a LiDAR or a depth camera gives a cloud of points; the next scan gives another cloud, from a slightly different pose
- pair every point with the **closest** point of the other scan; find the rotation and translation that align the pairs best (least squares again, this one even has a closed form); move the scan and **repeat**: as the scans get closer, the pairs get better
- it converges only from a close enough start: the previous pose, or the odometry, provides it

> The result is the motion between the two scans: an odometry term $\mathbf{z}_t$, or a loop closure, for the SLAM cost.

Iterative closest point (ICP), Besl & McKay 1992.

SLAM in the robot's stack



- the controller needs the pose, the planner needs the map: SLAM provides both
- map once, localise for months: build the map with SLAM, then localise against it (cheaper, no drift)
- where it runs today: vacuum and warehouse robots, drones and headsets, cars, planetary rovers

Summary

- **SfM**: features → matched pairs → geometric verification; then one image at a time: PnP, triangulation and bundle adjustment (COLMAP)
- **bundle adjustment**: minimise the reprojection error over all cameras and points at once: more constraints than unknowns, so least squares; nonlinear, so linearise, solve, move, iterate; the matrix is sparse
- NeRFs and Gaussian splatting build on the cameras and points that structure from motion gives
- **SLAM**: the same least squares on a moving robot: unknowns = poses + landmarks; prior + odometry + observation terms, each weighted by its noise
- a loop closure is one more measurement between now and then: it corrects the whole lap at once
- a front end per sensor turns data into measurements; one back end optimises; keyframes keep it real-time

> Write down what each measurement should be given the unknowns, subtract what it was, square, weight, sum; then linearise and solve, over a sparse graph. Reconstruction and SLAM are both instances of this.

Further reading: Szeliski, Computer Vision (2nd ed.), ch. 11; Hartley & Zisserman, Multiple View Geometry, ch. 18; Triggs et al., Bundle Adjustment: A Modern Synthesis (1999); Schönberger & Frahm, Structure-from-Motion Revisited (COLMAP, 2016); Grisetti et al., A Tutorial on Graph-Based SLAM (2010); Cadena et al., Past, Present, and Future of SLAM (2016); Dellaert & Kaess, Factor Graphs for Robot Perception (2017).

Feedback



Please spend a minute: your feedback helps us improve the next lecture.

`tinyurl.com/mimuw-rc-feedback`

Anonymous, no login, three questions.